

ERC Token Standards 学习文档

目录

1. [ERC20 - 可替代代币标准](#)
2. [ERC721 - 不可替代代币标准](#)
3. [ERC1155 - 多代币标准](#)
4. [标准对比](#)
5. [实际应用场景](#)

ERC20 - 可替代代币标准

概述

ERC20是以太坊上最广泛使用的代币标准，定义了可替代代币的基本接口。所有ERC20代币都是可互换的，每个代币的价值和功能完全相同。这个标准由Fabian Vogelsteller在2015年提出，为以太坊生态系统中的代币创建提供了统一的标准。

什么是可替代代币？

可替代代币（Fungible Token）是指每个代币在功能上完全相同，可以互相替换的代币。就像现实世界中的货币一样，一张100元的人民币和另一张100元的人民币在价值上是完全相同的，可以互相替换。在区块链世界中，所有的USDT代币都是相同的，1个USDT总是等于1个USDT。

ERC20标准的重要性

ERC20标准的重要性在于它提供了一套标准化的接口，使得不同的代币可以在同一个生态系统中无缝交互。如果没有这个标准，每个代币合约都需要自定义接口，这将导致：

- 钱包无法统一支持所有代币
- 交易所需要为每种代币开发不同的集成代码
- DeFi协议无法通用地处理不同代币
- 用户体验极差

核心功能详解

必需函数

ERC20标准定义了6个必需函数，这些函数构成了代币的基本功能框架：

```
// 总供应量 - 返回代币的总发行量
function totalSupply() public view returns (uint256)

// 账户余额 - 查询指定地址的代币余额
function balanceOf(address account) public view returns (uint256)

// 转账 - 从调用者账户向指定地址转账
```

```
function transfer(address to, uint256 amount) public returns (bool)

// 授权转账 - 从授权者账户向指定地址转账（需要预先授权）
function transferFrom(address from, address to, uint256 amount) public returns (bool)

// 授权额度 - 授权指定地址使用调用者账户的代币
function approve(address spender, uint256 amount) public returns (bool)

// 查询授权额度 - 查询授权者授权给被授权者的代币数量
function allowance(address owner, address spender) public view returns (uint256)
```

事件系统

ERC20标准定义了2个必需事件，用于记录代币的重要操作：

```
// 转账事件 - 当发生代币转账时触发
event Transfer(address indexed from, address indexed to, uint256 value);

// 授权事件 - 当发生代币授权时触发
event Approval(address indexed owner, address indexed spender, uint256 value);
```

授权机制详解

ERC20的授权机制是其核心特性之一，它允许用户授权第三方（如DApp或智能合约）代表自己进行代币操作。这个机制的工作流程如下：

1. **授权阶段**：用户调用 `approve` 函数，授权指定地址使用一定数量的代币
2. **使用阶段**：被授权的地址调用 `transferFrom` 函数，从用户账户转账代币
3. **查询阶段**：任何地址都可以通过 `allowance` 函数查询授权额度

这种机制在DeFi应用中非常重要，例如：

- 用户授权Uniswap使用USDT来购买ETH
- 用户授权Compound使用USDC来提供流动性
- 用户授权某个游戏合约使用游戏代币

完整实现示例

合约结构说明

下面的代码展示了一个完整的ERC20代币实现。在阅读代码之前，让我们先了解合约的整体结构：

1. **状态变量**：存储代币的基本信息（名称、符号、精度、总供应量）和账户数据（余额、授权额度）
2. **构造函数**：初始化代币的基本参数，并将所有代币分配给合约部署者
3. **转账函数**：实现代币的转账功能，包括直接转账和授权转账
4. **授权函数**：实现代币的授权功能，允许第三方代表用户操作代币

代码实现流程

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
```

```

contract ERC20Token {
    // 代币基本信息
    string public name;           // 代币名称, 如"Tether USD"
    string public symbol;         // 代币符号, 如"USDT"
    uint8 public decimals;        // 代币精度, 通常为18或6
    uint256 public totalSupply;   // 代币总供应量

    // 账户数据存储
    mapping(address => uint256) public balanceOf; // 地址到余额的映射
    mapping(address => mapping(address => uint256)) public allowance; // 授权额度映射

    // 事件定义
    event Transfer(address indexed from, address indexed to, uint256 value);
    event Approval(address indexed owner, address indexed spender, uint256 value);

    // 构造函数: 初始化代币参数
    constructor(
        string memory _name,
        string memory _symbol,
        uint8 _decimals,
        uint256 _totalSupply
    ) {
        name = _name;
        symbol = _symbol;
        decimals = _decimals;
        // 根据精度计算实际总供应量
        totalSupply = _totalSupply * 10**_decimals;
        // 将所有代币分配给合约部署者
        balanceOf[msg.sender] = totalSupply;
        // 触发转账事件, 表示代币从零地址转移到部署者
        emit Transfer(address(0), msg.sender, totalSupply);
    }

    // 直接转账函数
    function transfer(address to, uint256 amount) public returns (bool) {
        // 安全检查: 不能向零地址转账
        require(to != address(0), "Transfer to zero address");
        // 安全检查: 发送者余额必须足够
        require(balanceOf[msg.sender] >= amount, "Insufficient balance");

        // 更新余额
        balanceOf[msg.sender] -= amount;
        balanceOf[to] += amount;

        // 触发转账事件
        emit Transfer(msg.sender, to, amount);
        return true;
    }

    // 授权转账函数 (第三方代表用户转账)

```

```

function transferFrom(address from, address to, uint256 amount) public returns (bool)
{
    // 安全检查：发送者和接收者都不能是零地址
    require(from != address(0), "Transfer from zero address");
    require(to != address(0), "Transfer to zero address");
    // 安全检查：发送者余额必须足够
    require(balanceOf[from] >= amount, "Insufficient balance");
    // 安全检查：调用者必须有足够的授权额度
    require(allowance[from][msg.sender] >= amount, "Insufficient allowance");

    // 更新余额和授权额度
    balanceOf[from] -= amount;
    balanceOf[to] += amount;
    allowance[from][msg.sender] -= amount;

    // 触发转账事件
    emit Transfer(from, to, amount);
    return true;
}

// 授权函数（允许第三方使用自己的代币）
function approve(address spender, uint256 amount) public returns (bool) {
    // 安全检查：不能授权给零地址
    require(spender != address(0), "Approve to zero address");

    // 设置授权额度
    allowance[msg.sender][spender] = amount;
    // 触发授权事件
    emit Approval(msg.sender, spender, amount);
    return true;
}
}

```

代码执行流程详解

1. 代币创建流程

- 部署合约时，构造函数被调用
- 设置代币的基本信息（名称、符号、精度）
- 计算总供应量（考虑精度）
- 将所有代币分配给部署者
- 触发Transfer事件记录代币创建

2. 直接转账流程

- 用户调用transfer函数
- 检查接收地址不为零地址
- 检查发送者余额是否足够
- 更新发送者和接收者的余额
- 触发Transfer事件记录转账

3. 授权转账流程

- 用户先调用approve函数授权
- 第三方调用transferFrom函数
- 检查所有必要的条件（地址、余额、授权额度）
- 更新余额和授权额度
- 触发Transfer事件记录转账

4. 授权流程

- 用户调用approve函数
- 检查被授权地址不为零地址
- 设置授权额度
- 触发Approval事件记录授权

常见应用场景

- 加密货币: 如USDT、USDC等稳定币
- 治理代币: 如UNI、COMP等DeFi治理代币
- 实用代币: 如游戏内货币、积分系统
- 证券代币: 代表公司股份的代币化证券

ERC721 - 不可替代代币标准

概述

ERC721是以太坊上第一个也是最重要的不可替代代币(NFT)标准，由William Entriken、Dieter Shirley、Jacob Evans和Nastassia Sachs在2018年提出。这个标准定义了NFT的基本接口，使得每个代币都有唯一的标识符，不可互换，通常用于代表独特的数字资产。

什么是不可替代代币？

不可替代代币（Non-Fungible Token，NFT）是指每个代币都是独一无二的，具有独特的属性和价值，不能互相替换的代币。就像现实世界中的艺术品一样，每幅画都是独特的，不能简单地用另一幅画来替换。在区块链世界中，每个NFT都有唯一的tokenId，即使它们来自同一个合约，也是完全不同的资产。

NFT的核心特性

1. **唯一性**：每个NFT都有唯一的标识符（tokenId）
2. **不可分割性**：NFT不能像ERC20代币那样分割成更小的单位
3. **可验证性**：通过区块链技术可以验证NFT的真实性和所有权
4. **可编程性**：NFT可以包含复杂的逻辑和元数据
5. **互操作性**：遵循ERC721标准的NFT可以在不同的平台和钱包中使用

ERC721标准的重要性

ERC721标准的重要性在于它开创了数字资产所有权的新模式：

- **数字稀缺性**：首次实现了数字资产的真正稀缺性
- **所有权证明**：提供了不可篡改的所有权记录
- **跨平台兼容**：标准化的接口使得NFT可以在不同平台间流通

- 创新应用：为数字艺术、游戏、身份认证等领域提供了新的可能性

核心功能详解

必需函数

ERC721标准定义了8个必需函数，这些函数构成了NFT的基本功能框架：

```
// 查询余额 - 返回指定地址拥有的NFT数量
function balanceOf(address owner) public view returns (uint256)

// 查询所有者 - 返回指定tokenId的当前所有者
function ownerOf(uint256 tokenId) public view returns (address)

// 安全转账 - 安全地将NFT从from转移到to，包含数据参数
function safeTransferFrom(address from, address to, uint256 tokenId, bytes calldata data)
external

// 转账 - 将NFT从from转移到to
function transferFrom(address from, address to, uint256 tokenId) external

// 授权 - 授权指定地址使用特定的NFT
function approve(address to, uint256 tokenId) external

// 设置操作员 - 授权或取消授权操作员管理所有NFT
function setApprovalForAll(address operator, bool approved) external

// 获取授权地址 - 查询特定NFT的授权地址
function getApproved(uint256 tokenId) public view returns (address)

// 查询是否为操作员 - 查询operator是否为owner的操作员
function isApprovedForAll(address owner, address operator) public view returns (bool)
```

事件系统

ERC721标准定义了3个必需事件，用于记录NFT的重要操作：

```
// 转账事件 - 当NFT发生转移时触发
event Transfer(address indexed from, address indexed to, uint256 indexed tokenId);

// 授权事件 - 当特定NFT被授权时触发
event Approval(address indexed owner, address indexed approved, uint256 indexed tokenId);

// 操作员授权事件 - 当设置操作员权限时触发
event ApprovalForAll(address indexed owner, address indexed operator, bool approved);
```

授权机制详解

ERC721的授权机制比ERC20更加灵活，提供了两种授权方式：

1. 单个NFT授权 (approve)

- 用户授权特定地址使用特定的NFT
- 授权只对指定的tokenId有效
- 适用于精确控制特定NFT的使用权限

2. 批量授权 (setApprovalForAll)

- 用户授权操作员管理其所有NFT
- 授权对所有当前和未来的NFT有效
- 适用于需要批量管理NFT的场景，如NFT市场

安全转账机制

ERC721引入了安全转账的概念，通过 `safeTransferFrom` 函数确保：

- 接收方是合约时，必须实现 `onERC721Received` 函数
- 防止NFT被发送到无法处理它们的合约中
- 提供额外的安全保障机制

完整实现示例

基础NFT合约实现

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/utils/Counters.sol";
import "@openzeppelin/contracts/utils/Strings.sol";

contract MyNFT is ERC721, Ownable {
    using Counters for Counters.Counter;
    using Strings for uint256;

    Counters.Counter private _tokenIdCounter;
    string private _baseTokenURI;
    uint256 public constant MAX_SUPPLY = 10000; // 最大供应量
    uint256 public constant MINT_PRICE = 0.08 ether; // 铸造价格

    // 元数据存储
    mapping(uint256 => string) private _tokenURIs;

    // 事件
    event TokenMinted(address indexed to, uint256 indexed tokenId);
    event BaseURIUpdated(string newBaseURI);

    constructor(string memory name, string memory symbol) ERC721(name, symbol) {}

    // 铸造函数
    function mint(address to) public payable returns (uint256) {
        require(_tokenIdCounter.current() < MAX_SUPPLY, "Max supply reached");
        require(msg.value >= MINT_PRICE, "Insufficient payment");
```

```

    uint256 tokenId = _tokenIdCounter.current();
    _tokenIdCounter.increment();
    _safeMint(to, tokenId);

    emit TokenMinted(to, tokenId);
    return tokenId;
}

// 批量铸造
function mintBatch(address to, uint256 quantity) public payable returns (uint256[]
memory) {
    require(quantity > 0 && quantity <= 10, "Invalid quantity");
    require(_tokenIdCounter.current() + quantity <= MAX_SUPPLY, "Exceeds max supply");
    require(msg.value >= MINT_PRICE * quantity, "Insufficient payment");

    uint256[] memory tokenIds = new uint256[](quantity);

    for (uint256 i = 0; i < quantity; i++) {
        uint256 tokenId = _tokenIdCounter.current();
        _tokenIdCounter.increment();
        _safeMint(to, tokenId);
        tokenIds[i] = tokenId;
        emit TokenMinted(to, tokenId);
    }

    return tokenIds;
}

// 设置基础URI
function setBaseURI(string memory baseURI) public onlyOwner {
    _baseTokenURI = baseURI;
    emit BaseURIUpdated(baseURI);
}

// 设置特定token的URI
function setTokenURI(uint256 tokenId, string memory tokenURI) public onlyOwner {
    require(!_exists(tokenId), "Token does not exist");
    _tokenURIs[tokenId] = tokenURI;
}

// 获取基础URI
function _baseURI() internal view override returns (string memory) {
    return _baseTokenURI;
}

// 获取token URI
function tokenURI(uint256 tokenId) public view override returns (string memory) {
    require(!_exists(tokenId), "Token does not exist");

    // 如果设置了特定URI, 使用特定URI
    if (bytes(_tokenURIs[tokenId]).length > 0) {

```



```

        return _tokenURIs[tokenId];
    }

    // 否则使用基础URI + tokenId
    return string(abi.encodePacked(_baseURI(), tokenId.toString()));
}

// 获取总供应量
function totalSupply() public view returns (uint256) {
    return _tokenIdCounter.current();
}

// 提取合约余额
function withdraw() public onlyOwner {
    uint256 balance = address(this).balance;
    require(balance > 0, "No balance to withdraw");

    (bool success, ) = payable(owner()).call{value: balance}("");
    require(success, "Withdrawal failed");
}
}

```

高级NFT合约实现（包含元数据和稀有度）

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/utils/Counters.sol";

contract AdvancedNFT is ERC721, Ownable {
    using Counters for Counters.Counter;

    Counters.Counter private _tokenIdCounter;

    // NFT属性结构
    struct NFTAttributes {
        string name;
        string description;
        string image;
        uint256 rarity; // 稀有度等级 1-5
        uint256 level; // 等级
        uint256 power; // 力量值
        bool isSpecial; // 是否为特殊NFT
    }

    // 存储每个NFT的属性
    mapping(uint256 => NFTAttributes) public tokenAttributes;

    // 稀有度分布

```

```

mapping(uint256 => uint256) public rarityCount;

// 事件
event NFTMinted(address indexed to, uint256 indexed tokenId, NFTAttributes
attributes);
event AttributesUpdated(uint256 indexed tokenId, NFTAttributes attributes);

constructor() ERC721("AdvancedNFT", "ANFT") {}

// 铸造NFT
function mintNFT(
    address to,
    string memory name,
    string memory description,
    string memory image,
    uint256 rarity,
    uint256 level,
    uint256 power,
    bool isSpecial
) public onlyOwner returns (uint256) {
    require(rarity >= 1 && rarity <= 5, "Invalid rarity");

    uint256 tokenId = _tokenIdCounter.current();
    _tokenIdCounter.increment();

    NFTAttributes memory attributes = NFTAttributes({
        name: name,
        description: description,
        image: image,
        rarity: rarity,
        level: level,
        power: power,
        isSpecial: isSpecial
    });

    tokenAttributes[tokenId] = attributes;
    rarityCount[rarity]++;

    _safeMint(to, tokenId);

    emit NFTMinted(to, tokenId, attributes);
    return tokenId;
}

// 更新NFT属性
function updateAttributes(
    uint256 tokenId,
    string memory name,
    string memory description,
    string memory image,
    uint256 rarity,
    uint256 level,

```

```

    uint256 power,
    bool isSpecial
) public onlyOwner {
    require(!_exists(tokenId), "Token does not exist");
    require(rarity >= 1 && rarity <= 5, "Invalid rarity");

    NFTAttributes memory oldAttributes = tokenAttributes[tokenId];
    rarityCount[oldAttributes.rarity]--;

    NFTAttributes memory newAttributes = NFTAttributes({
        name: name,
        description: description,
        image: image,
        rarity: rarity,
        level: level,
        power: power,
        isSpecial: isSpecial
    });

    tokenAttributes[tokenId] = newAttributes;
    rarityCount[rarity]++;

    emit AttributesUpdated(tokenId, newAttributes);
}

// 获取NFT属性
function getAttributes(uint256 tokenId) public view returns (NFTAttributes memory) {
    require(!_exists(tokenId), "Token does not exist");
    return tokenAttributes[tokenId];
}

// 获取稀有度统计
function getRarityStats() public view returns (uint256[6] memory) {
    uint256[6] memory stats;
    for (uint256 i = 1; i <= 5; i++) {
        stats[i] = rarityCount[i];
    }
    return stats;
}

// 重写tokenURI函数
function tokenURI(uint256 tokenId) public view override returns (string memory) {
    require(!_exists(tokenId), "Token does not exist");

    NFTAttributes memory attributes = tokenAttributes[tokenId];

    // 构建JSON元数据
    string memory json = string(abi.encodePacked(
        '{"name":"' , attributes.name, '",' ,
        '"description":"' , attributes.description, '",' ,
        '"image":"' , attributes.image, '",' ,
        '"attributes":[' ,

```

```

        '{"trait_type":"Rarity","value":', uint2str(attributes.rarity), '},',
        '{"trait_type":"Level","value":', uint2str(attributes.level), '},',
        '{"trait_type":"Power","value":', uint2str(attributes.power), '},',
        '{"trait_type":"Special","value":', attributes.isSpecial ? 'true' : 'false',
    '}',
    ']]'
));

return string(abi.encodePacked("data:application/json;base64,", encode(json)));
}

// 辅助函数: 将uint转换为string
function uint2str(uint256 _i) internal pure returns (string memory) {
    if (_i == 0) {
        return "0";
    }
    uint256 j = _i;
    uint256 len;
    while (j != 0) {
        len++;
        j /= 10;
    }
    bytes memory bstr = new bytes(len);
    uint256 k = len;
    while (_i != 0) {
        k = k - 1;
        uint8 temp = (48 + uint8(_i - _i / 10 * 10));
        bytes1 b1 = bytes1(temp);
        bstr[k] = b1;
        _i /= 10;
    }
    return string(bstr);
}

// 辅助函数: Base64编码
function encode(string memory _str) internal pure returns (string memory) {
    bytes memory data = bytes(_str);
    if (data.length == 0) return "";

    string memory table =
"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/";

    uint256 encodedLen = 4 * ((data.length + 2) / 3);
    string memory result = new string(encodedLen + 32);

    assembly {
        let tablePtr := add(table, 1)
        let resultPtr := add(result, 32)

        for {
            let i := 0
        } lt(i, mload(data)) {

```

```

        i := add(i, 3)
    } {
        let input := and(mload(add(data, add(32, i))), 0xffffffff)

        let out := mload(add(tablePtr, and(shr(250, input), 0x3F)))
        out := shl(8, out)
        out := add(out, and(mload(add(tablePtr, and(shr(244, input), 0x3F))),
0xFF))

        out := shl(8, out)
        out := add(out, and(mload(add(tablePtr, and(shr(238, input), 0x3F))),
0xFF))

        out := shl(8, out)
        out := add(out, and(mload(add(tablePtr, and(shr(232, input), 0x3F))),
0xFF))

        out := shl(224, out)

        mstore(resultPtr, out)
        resultPtr := add(resultPtr, 4)
    }

    switch mod(mload(data), 3)
    case 1 {
        mstore(sub(resultPtr, 2), shl(240, 0x3d3d))
    }
    case 2 {
        mstore(sub(resultPtr, 1), shl(248, 0x3d))
    }
}

return result;
}
}

```

常见应用场景

1. 数字艺术品

- **CryptoPunks**: 10,000个独特的像素艺术头像
- **Bored Ape Yacht Club**: 10,000个猿猴头像NFT
- **Art Blocks**: 算法生成的艺术作品
- **SuperRare**: 数字艺术市场平台

2. 游戏道具

- **Axie Infinity**: 游戏中的宠物和道具
- **CryptoKitties**: 虚拟猫咪收集游戏
- **The Sandbox**: 虚拟世界中的土地和物品
- **Decentraland**: 虚拟世界中的土地和建筑

3. 域名系统

- **ENS (Ethereum Name Service):** 以太坊域名服务
- **Unstoppable Domains:** 去中心化域名
- **Handshake:** 去中心化域名协议

4. 身份认证

- **POAP (Proof of Attendance Protocol):** 出席证明协议
- **Gitcoin Passport:** 去中心化身份验证
- **BrightID:** 社交身份网络

5. 音乐和媒体

- **Audius:** 去中心化音乐平台
- **Async Art:** 可编程艺术平台
- **Foundation:** 创作者经济平台

6. 体育和收藏品

- **NBA Top Shot:** NBA精彩瞬间收藏
- **Sorare:** 足球球员卡游戏
- **Topps:** 数字交易卡

ERC721的优缺点

优点

1. **标准化:** 提供了统一的NFT接口标准
2. **互操作性:** 可以在不同平台间自由流通
3. **安全性:** 内置安全转账机制
4. **灵活性:** 支持复杂的元数据和属性
5. **可扩展性:** 可以添加自定义功能

缺点

1. **Gas费用高:** 每个NFT操作都需要支付Gas费
2. **存储成本:** 元数据存储成本较高
3. **批量操作复杂:** 批量转账需要多次交易
4. **不可分割:** 无法像ERC20那样分割成更小单位

ERC1155 - 多代币标准

概述

ERC1155是一个由Enjin团队提出的革命性的多代币标准，可以在单个合约中管理多种类型的代币，包括可替代和不可替代代币。它结合了ERC20和ERC721的优势，提供了更高的效率和灵活性，特别适合游戏、DeFi和复杂资产管理场景。

核心特性

1. 多代币支持

- 可替代代币: 如游戏货币、积分系统
- 不可替代代币: 如独特道具、艺术品
- 半可替代代币: 如限量版但可分割的资产

2. 批量操作

- 批量转账: 一次交易转移多种代币
- 批量查询: 高效查询多个代币余额
- 批量授权: 统一管理操作权限

3. Gas效率优化

- 单次交易: 减少交易次数和Gas消耗
- 状态更新: 优化存储和计算成本
- 事件优化: 减少事件日志开销

核心功能详解

必需函数

```
// 查询单个代币余额
function balanceOf(address account, uint256 id) external view returns (uint256)

// 批量查询余额 - 核心优势
function balanceOfBatch(address[] calldata accounts, uint256[] calldata ids)
    external view returns (uint256[] memory)

// 设置全局授权 - 简化权限管理
function setApprovalForAll(address operator, bool approved) external

// 查询授权状态
function isApprovedForAll(address account, address operator) external view returns (bool)

// 安全转账 - 支持回调
function safeTransferFrom(
    address from,
    address to,
    uint256 id,
    uint256 amount,
    bytes calldata data
) external

// 安全批量转账 - 核心功能
function safeBatchTransferFrom(
    address from,
    address to,
    uint256[] calldata ids,
    uint256[] calldata amounts,
    bytes calldata data
) external
```

事件系统

```
// 单个代币转账事件
event TransferSingle(
    address indexed operator,
    address indexed from,
    address indexed to,
    uint256 id,
    uint256 value
);

// 批量转账事件 - 高效记录
event TransferBatch(
    address indexed operator,
    address indexed from,
    address indexed to,
    uint256[] ids,
    uint256[] values
);

// 授权状态变更事件
event ApprovalForAll(
    address indexed account,
    address indexed operator,
    bool approved
);

// URI更新事件 - 元数据管理
event URI(string value, uint256 indexed id);
```

高级实现示例

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "@openzeppelin/contracts/token/ERC1155/ERC1155.sol";
import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/security/Pausable.sol";
import "@openzeppelin/contracts/security/ReentrancyGuard.sol";

contract AdvancedMultiToken is ERC1155, Ownable, Pausable, ReentrancyGuard {
    // 代币元数据
    struct TokenInfo {
        string name;
        string symbol;
        uint256 maxSupply;
        uint256 currentSupply;
        bool isFungible;
        bool isPaused;
    }
}
```



```

// 状态变量
mapping(uint256 => TokenInfo) public tokenInfo;
mapping(uint256 => string) private _tokenURIs;
mapping(address => bool) public minters;

// 事件
event TokenCreated(uint256 indexed id, string name, string symbol, bool isFungible);
event TokenMinted(uint256 indexed id, address indexed to, uint256 amount);
event TokenBurned(uint256 indexed id, address indexed from, uint256 amount);

constructor() ERC1155("") {}

// 创建新代币类型
function createToken(
    uint256 id,
    string memory name,
    string memory symbol,
    uint256 maxSupply,
    bool isFungible,
    string memory tokenURI
) external onlyOwner {
    require(tokenInfo[id].maxSupply == 0, "Token already exists");

    tokenInfo[id] = TokenInfo({
        name: name,
        symbol: symbol,
        maxSupply: maxSupply,
        currentSupply: 0,
        isFungible: isFungible,
        isPaused: false
    });

    _tokenURIs[id] = tokenURI;
    emit TokenCreated(id, name, symbol, isFungible);
    emit URI(tokenURI, id);
}

// 铸造代币
function mint(
    address to,
    uint256 id,
    uint256 amount,
    bytes memory data
) external onlyMinter nonReentrant {
    require(tokenInfo[id].maxSupply > 0, "Token does not exist");
    require(!tokenInfo[id].isPaused, "Token minting paused");
    require(
        tokenInfo[id].currentSupply + amount <= tokenInfo[id].maxSupply,
        "Exceeds max supply"
    );
}

```

```

        tokenInfo[id].currentSupply += amount;
        _mint(to, id, amount, data);
        emit TokenMinted(id, to, amount);
    }

    // 批量铸造
    function mintBatch(
        address to,
        uint256[] memory ids,
        uint256[] memory amounts,
        bytes memory data
    ) external onlyMinter nonReentrant {
        require(ids.length == amounts.length, "Arrays length mismatch");

        for (uint256 i = 0; i < ids.length; i++) {
            require(tokenInfo[ids[i]].maxSupply > 0, "Token does not exist");
            require(!tokenInfo[ids[i]].isPaused, "Token minting paused");
            require(
                tokenInfo[ids[i]].currentSupply + amounts[i] <=
tokenInfo[ids[i]].maxSupply,
                "Exceeds max supply"
            );

            tokenInfo[ids[i]].currentSupply += amounts[i];
        }

        _mintBatch(to, ids, amounts, data);

        for (uint256 i = 0; i < ids.length; i++) {
            emit TokenMinted(ids[i], to, amounts[i]);
        }
    }

    // 销毁代币
    function burn(
        address from,
        uint256 id,
        uint256 amount
    ) external {
        require(
            from == msg.sender || isApprovedForAll(from, msg.sender),
            "Not authorized"
        );

        tokenInfo[id].currentSupply -= amount;
        _burn(from, id, amount);
        emit TokenBurned(id, from, amount);
    }

    // 批量销毁
    function burnBatch(
        address from,

```

```

    uint256[] memory ids,
    uint256[] memory amounts
) external {
    require(
        from == msg.sender || isApprovedForAll(from, msg.sender),
        "Not authorized"
    );

    for (uint256 i = 0; i < ids.length; i++) {
        tokenInfo[ids[i]].currentSupply -= amounts[i];
    }

    _burnBatch(from, ids, amounts);

    for (uint256 i = 0; i < ids.length; i++) {
        emit TokenBurned(ids[i], from, amounts[i]);
    }
}

// 更新代币URI
function setURI(uint256 tokenId, string memory tokenURI) external onlyOwner {
    _tokenURIs[tokenId] = tokenURI;
    emit URI(tokenURI, tokenId);
}

// 获取代币URI
function uri(uint256 tokenId) public view override returns (string memory) {
    return _tokenURIs[tokenId];
}

// 暂停/恢复代币
function pauseToken(uint256 id) external onlyOwner {
    tokenInfo[id].isPaused = true;
}

function unpauseToken(uint256 id) external onlyOwner {
    tokenInfo[id].isPaused = false;
}

// 设置铸造者权限
function setMinter(address minter, bool status) external onlyOwner {
    minters[minter] = status;
}

// 修改器
modifier onlyMinter() {
    require(minters[msg.sender] || msg.sender == owner(), "Not a minter");
    _;
}

// 重写转账函数以支持暂停
function _beforeTokenTransfer(

```

```

        address operator,
        address from,
        address to,
        uint256[] memory ids,
        uint256[] memory amounts,
        bytes memory data
    ) internal override whenNotPaused {
        super._beforeTokenTransfer(operator, from, to, ids, amounts, data);
    }
}

```

实际应用场景详解

1. 游戏生态系统

```

// 游戏道具和货币管理
contract GameEconomy is ERC1155 {
    // 代币ID定义
    uint256 public constant GOLD_COIN = 1;           // 可替代 - 游戏货币
    uint256 public constant SILVER_COIN = 2;         // 可替代 - 游戏货币
    uint256 public constant SWORD = 1001;           // 不可替代 - 武器
    uint256 public constant SHIELD = 1002;          // 不可替代 - 防具
    uint256 public constant POTION = 2001;          // 可替代 - 消耗品

    // 游戏特定功能
    function tradeItems(
        address from,
        address to,
        uint256[] memory giveIds,
        uint256[] memory giveAmounts,
        uint256[] memory takeIds,
        uint256[] memory takeAmounts
    ) external {
        // 先转移要给出的物品
        _safeBatchTransferFrom(from, to, giveIds, giveAmounts, "");
        // 再转移要获得的物品
        _safeBatchTransferFrom(to, from, takeIds, takeAmounts, "");
    }
}

```

2. DeFi协议集成

```

// DeFi多代币管理
contract DeFiVault is ERC1155 {
    // 代币类型
    uint256 public constant LP_TOKEN = 1;           // 流动性代币
    uint256 public constant REWARD_TOKEN = 2;       // 奖励代币
    uint256 public constant GOVERNANCE_TOKEN = 3;    // 治理代币

    // 质押功能

```

```

function stake(
    uint256[] memory tokenIds,
    uint256[] memory amounts
) external {
    // 转移代币到合约
    _safeBatchTransferFrom(msg.sender, address(this), tokenIds, amounts, "");
    // 记录质押信息
    // 发放质押奖励代币
}
}

```

3. NFT市场批量交易

```

// 批量NFT交易
contract BatchNFTMarket is ERC1155 {
    struct Listing {
        address seller;
        uint256[] tokenIds;
        uint256[] amounts;
        uint256 price;
        bool active;
    }

    mapping(uint256 => Listing) public listings;

    function createBatchListing(
        uint256[] memory tokenIds,
        uint256[] memory amounts,
        uint256 price
    ) external {
        // 转移代币到合约
        _safeBatchTransferFrom(msg.sender, address(this), tokenIds, amounts, "");
        // 创建批量出售列表
    }

    function buyBatchListing(uint256 listingId) external payable {
        Listing storage listing = listings[listingId];
        require(listing.active, "Listing not active");
        require(msg.value >= listing.price, "Insufficient payment");

        // 转移代币给买家
        _safeBatchTransferFrom(
            address(this),
            msg.sender,
            listing.tokenIds,
            listing.amounts,
            ""
        );

        // 支付给卖家
        payable(listing.seller).transfer(listing.price);
    }
}

```

```
        listing.active = false;
    }
}
```

优势与挑战

优势

1. **Gas效率**: 批量操作大幅降低Gas消耗
2. **灵活性**: 支持多种代币类型
3. **可扩展性**: 易于添加新的代币类型
4. **互操作性**: 标准化的接口便于集成
5. **开发效率**: 减少合约部署和维护成本

挑战

1. **复杂性**: 比ERC20/ERC721更复杂
2. **兼容性**: 需要前端和工具支持
3. **学习曲线**: 开发者需要理解新概念
4. **存储成本**: 元数据存储可能较高

最佳实践

1. 代币ID设计

```
// 推荐的ID设计模式
uint256 public constant FUNGIBLE_START = 1;
uint256 public constant FUNGIBLE_END = 1000;
uint256 public constant NFT_START = 1001;
uint256 public constant NFT_END = 1000000;

function isFungible(uint256 id) public pure returns (bool) {
    return id >= FUNGIBLE_START && id <= FUNGIBLE_END;
}
```

2. 批量操作优化

```
// 高效的批量查询
function getBatchBalances(
    address account,
    uint256[] memory ids
) external view returns (uint256[] memory) {
    uint256[] memory balances = new uint256[](ids.length);
    for (uint256 i = 0; i < ids.length; i++) {
        balances[i] = balanceOf(account, ids[i]);
    }
    return balances;
}
```

3. 元数据管理

```
// 动态URI生成
function uri(uint256 tokenId) public view override returns (string memory) {
    return string(abi.encodePacked(baseURI, tokenId.toString()));
}
```

常见应用场景

- 游戏平台: 管理多种游戏道具、货币和装备
- DeFi协议: 统一管理LP代币、治理代币和奖励代币
- NFT市场: 支持批量交易和拍卖
- 供应链管理: 跟踪多种产品类型和批次
- 数字收藏: 管理系列收藏品和限量版
- 企业应用: 管理员工积分、证书和权限

标准对比分析

功能特性对比

特性	ERC20	ERC721	ERC1155
代币类型	可替代	不可替代	多代币
唯一性	无	有	可选
批量操作	无	无	有
Gas效率	中等	低	高
灵活性	低	中等	高
学习曲线	简单	中等	复杂
部署成本	低	中等	低
维护成本	低	高	低

使用场景对比

ERC20 适用场景

- 货币系统: 稳定币、治理代币
- 积分系统: 用户积分、奖励代币
- DeFi协议: 流动性代币、质押代币
- 简单应用: 不需要复杂功能的场景

ERC721 适用场景

- 数字艺术: 独特的艺术品和收藏品
- 游戏道具: 独特的武器、装备
- 身份认证: 数字身份、证书
- 房地产: 虚拟土地、房产

ERC1155 适用场景

- 游戏生态: 多种道具和货币
- DeFi协议: 多种代币类型管理
- NFT市场: 批量交易和拍卖
- 企业应用: 复杂的资产管理

Gas消耗对比

转账操作Gas消耗

```
// ERC20 转账
function transfer(address to, uint256 amount) external
// Gas消耗: ~21,000 + 存储更新

// ERC721 转账
function transferFrom(address from, address to, uint256 tokenId) external
// Gas消耗: ~45,000 + 存储更新

// ERC1155 单个转账
function safeTransferFrom(address from, address to, uint256 id, uint256 amount, bytes
calldata data) external
// Gas消耗: ~35,000 + 存储更新

// ERC1155 批量转账 (5个代币)
function safeBatchTransferFrom(address from, address to, uint256[] calldata ids, uint256[]
calldata amounts, bytes calldata data) external
// Gas消耗: ~65,000 (平均每个代币13,000)
```

查询操作Gas消耗

```
// ERC20 余额查询
function balanceOf(address account) external view returns (uint256)
// Gas消耗: ~2,100

// ERC721 余额查询
function balanceOf(address owner) external view returns (uint256)
// Gas消耗: ~2,100

// ERC1155 单个余额查询
function balanceOf(address account, uint256 id) external view returns (uint256)
// Gas消耗: ~2,100

// ERC1155 批量余额查询 (5个代币)
```



```
function balanceOfBatch(address[] calldata accounts, uint256[] calldata ids) external view
returns (uint256[] memory)
// Gas消耗: ~8,000 (平均每个代币1,600)
```

开发复杂度对比

ERC20 开发复杂度

```
// 简单实现 - 约50行代码
contract SimpleToken is ERC20 {
    constructor() ERC20("Token", "TKN") {
        _mint(msg.sender, 1000000 * 10**18);
    }
}
```

// 功能需求

- ☒ 基本转账功能
- ☒ 授权机制
- ☒ 事件记录
- ☒ 批量操作
- ☒ 元数据管理

ERC721 开发复杂度

```
// 标准实现 - 约200行代码
contract NFT is ERC721 {
    mapping(uint256 => string) private _tokenURIs;

    function mint(address to, uint256 tokenId) external {
        _mint(to, tokenId);
    }

    function setTokenURI(uint256 tokenId, string memory tokenURI) external {
        _tokenURIs[tokenId] = tokenURI;
    }
}
```

// 功能需求

- ☒ 唯一性保证
- ☒ 元数据管理
- ☒ 授权机制
- ☒ 批量操作
- ☒ 可替代性

ERC1155 开发复杂度

```
// 完整实现 - 约400行代码
contract MultiToken is ERC1155 {
    mapping(uint256 => TokenInfo) public tokenInfo;
    mapping(uint256 => string) private _tokenURIs;
```

```

struct TokenInfo {
    string name;
    uint256 maxSupply;
    bool isFungible;
}

function createToken(uint256 id, string memory name, uint256 maxSupply, bool
isFungible) external {
    // 复杂的代币创建逻辑
}

function mintBatch(address to, uint256[] memory ids, uint256[] memory amounts)
external {
    // 批量铸造逻辑
}
}

// 功能需求
- ☒ 多代币支持
- ☒ 批量操作
- ☒ 元数据管理
- ☒ 灵活配置
- ☒ 高效存储

```

选择建议

选择ERC20的情况

- 需要简单的可替代代币
- 预算有限，需要快速部署
- 团队对区块链开发经验有限
- 应用场景相对简单

选择ERC721的情况

- 需要保证代币的唯一性
- 主要处理数字艺术品或收藏品
- 不需要批量操作功能
- 对Gas效率要求不高

选择ERC1155的情况

- 需要管理多种代币类型
- 对Gas效率有较高要求
- 需要批量操作功能
- 有足够的开发资源
- 应用场景复杂多样

迁移策略

从ERC20到ERC1155

```
// 迁移合约示例
contract TokenMigration {
    ERC20 public oldToken;
    ERC1155 public newToken;

    function migrate(uint256 amount) external {
        // 销毁旧代币
        oldToken.transferFrom(msg.sender, address(this), amount);
        oldToken.burn(amount);

        // 铸造新代币
        newToken.mint(msg.sender, TOKEN_ID, amount, "");
    }
}
```

从ERC721到ERC1155

```
// NFT迁移示例
contract NFTMigration {
    ERC721 public oldNFT;
    ERC1155 public newToken;

    function migrate(uint256 tokenId) external {
        // 转移NFT所有权
        oldNFT.transferFrom(msg.sender, address(this), tokenId);

        // 铸造新的ERC1155代币
        newToken.mint(msg.sender, tokenId, 1, "");
    }
}
```

实际应用场景详解

ERC20 应用场景

1. 稳定币实现

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/security/Pausable.sol";

contract USDT is ERC20, Ownable, Pausable {
    // 储备金管理
```

```

mapping(address => uint256) public reserves;
uint256 public totalReserves;

// 事件
event Mint(address indexed to, uint256 amount, uint256 reserveAmount);
event Burn(address indexed from, uint256 amount, uint256 reserveAmount);
event ReserveDeposit(address indexed from, uint256 amount);
event ReserveWithdraw(address indexed to, uint256 amount);

constructor() ERC20("Tether USD", "USDT") {
    _mint(msg.sender, 1000000000 * 10**6); // 10亿USDT
}

// 铸造稳定币 (需要存入储备金)
function mint(address to, uint256 amount, uint256 reserveAmount) external onlyOwner {
    require(reserveAmount >= amount, "Insufficient reserve ratio");

    _mint(to, amount);
    reserves[msg.sender] += reserveAmount;
    totalReserves += reserveAmount;

    emit Mint(to, amount, reserveAmount);
}

// 销毁稳定币 (释放储备金)
function burn(address from, uint256 amount, uint256 reserveAmount) external onlyOwner
{
    require(balanceOf(from) >= amount, "Insufficient balance");
    require(reserves[msg.sender] >= reserveAmount, "Insufficient reserves");

    _burn(from, amount);
    reserves[msg.sender] -= reserveAmount;
    totalReserves -= reserveAmount;

    emit Burn(from, amount, reserveAmount);
}

// 存入储备金
function depositReserve() external payable {
    reserves[msg.sender] += msg.value;
    totalReserves += msg.value;
    emit ReserveDeposit(msg.sender, msg.value);
}

// 提取储备金
function withdrawReserve(uint256 amount) external {
    require(reserves[msg.sender] >= amount, "Insufficient reserves");

    reserves[msg.sender] -= amount;
    totalReserves -= amount;
    payable(msg.sender).transfer(amount);
}

```

```

        emit ReserveWithdraw(msg.sender, amount);
    }

    // 暂停功能
    function pause() external onlyOwner {
        _pause();
    }

    function unpause() external onlyOwner {
        _unpause();
    }

    // 重写转账函数以支持暂停
    function _beforeTokenTransfer(
        address from,
        address to,
        uint256 amount
    ) internal override whenNotPaused {
        super._beforeTokenTransfer(from, to, amount);
    }
}

```

2. 治理代币实现

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

contract GovernanceToken is ERC20, Ownable {
    // 投票相关
    mapping(address => uint256) public votingPower;
    mapping(address => address) public delegates;
    mapping(address => uint256) public delegatedVotes;

    // 提案相关
    struct Proposal {
        uint256 id;
        address proposer;
        string description;
        uint256 startBlock;
        uint256 endBlock;
        uint256 forVotes;
        uint256 againstVotes;
        bool executed;
        bool canceled;
    }

    mapping(uint256 => Proposal) public proposals;
    mapping(uint256 => mapping(address => bool)) public hasVoted;
}

```

```

uint256 public proposalCount;

// 事件
event DelegateChanged(address indexed delegator, address indexed fromDelegate, address indexed toDelegate);
event DelegateVotesChanged(address indexed delegate, uint256 previousBalance, uint256 newBalance);
event ProposalCreated(uint256 indexed proposalId, address indexed proposer, string description);
event VoteCast(address indexed voter, uint256 indexed proposalId, bool support, uint256 votes);
event ProposalExecuted(uint256 indexed proposalId);

constructor() ERC20("Governance Token", "GOV") {
    _mint(msg.sender, 1000000 * 10**18); // 100万代币
}

// 委托投票权
function delegate(address delegatee) external {
    address currentDelegate = delegates[msg.sender];
    require(delegatee != currentDelegate, "Already delegated to this address");

    delegates[msg.sender] = delegatee;

    if (currentDelegate != address(0)) {
        uint256 oldVotes = delegatedVotes[currentDelegate];
        delegatedVotes[currentDelegate] = oldVotes - balanceOf(msg.sender);
        emit DelegateVotesChanged(currentDelegate, oldVotes, delegatedVotes[currentDelegate]);
    }

    if (delegatee != address(0)) {
        uint256 newVotes = delegatedVotes[delegatee] + balanceOf(msg.sender);
        delegatedVotes[delegatee] = newVotes;
        emit DelegateVotesChanged(delegatee, delegatedVotes[delegatee] - balanceOf(msg.sender), newVotes);
    }

    emit DelegateChanged(msg.sender, currentDelegate, delegatee);
}

// 创建提案
function propose(string memory description) external returns (uint256) {
    require(balanceOf(msg.sender) >= 1000 * 10**18, "Insufficient balance to propose");

    proposalCount++;
    uint256 proposalId = proposalCount;

    proposals[proposalId] = Proposal({
        id: proposalId,
        proposer: msg.sender,

```

```

        description: description,
        startBlock: block.number,
        endBlock: block.number + 17280, // 约3天
        forVotes: 0,
        againstVotes: 0,
        executed: false,
        canceled: false
    });

    emit ProposalCreated(proposalId, msg.sender, description);
    return proposalId;
}

// 投票
function vote(uint256 proposalId, bool support) external {
    Proposal storage proposal = proposals[proposalId];
    require(block.number >= proposal.startBlock, "Voting not started");
    require(block.number <= proposal.endBlock, "Voting ended");
    require(!hasVoted[proposalId][msg.sender], "Already voted");

    uint256 votes = balanceOf(msg.sender);
    hasVoted[proposalId][msg.sender] = true;

    if (support) {
        proposal.forVotes += votes;
    } else {
        proposal.againstVotes += votes;
    }

    emit VoteCast(msg.sender, proposalId, support, votes);
}

// 执行提案
function executeProposal(uint256 proposalId) external {
    Proposal storage proposal = proposals[proposalId];
    require(block.number > proposal.endBlock, "Voting not ended");
    require(!proposal.executed, "Proposal already executed");
    require(!proposal.canceled, "Proposal canceled");
    require(proposal.forVotes > proposal.againstVotes, "Proposal not passed");

    proposal.executed = true;
    emit ProposalExecuted(proposalId);

    // 这里可以添加具体的执行逻辑
}

// 获取投票权
function getVotes(address account) external view returns (uint256) {
    return delegatedVotes[account];
}
}

```

ERC721 应用场景

1. 数字艺术品NFT

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "@openzeppelin/contracts/token/ERC721/extensions/ERC721URIStorage.sol";
import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/security/ReentrancyGuard.sol";

contract ArtNFT is ERC721URIStorage, Ownable, ReentrancyGuard {
    // 艺术品信息
    struct Artwork {
        string title;
        string artist;
        uint256 price;
        bool forSale;
        uint256 royaltyPercentage;
        address creator;
    }

    // 状态变量
    mapping(uint256 => Artwork) public artworks;
    mapping(address => uint256[]) public artistArtworks;
    mapping(address => uint256) public artistRoyalties;

    uint256 public totalArtworks;
    uint256 public constant MAX_ROYALTY = 1000; // 10%

    // 事件
    event ArtworkCreated(uint256 indexed tokenId, string title, string artist, uint256 price);
    event ArtworkSold(uint256 indexed tokenId, address indexed buyer, uint256 price);
    event RoyaltyPaid(uint256 indexed tokenId, address indexed artist, uint256 amount);

    constructor() ERC721("Digital Art", "ART") {}

    // 创建艺术品
    function createArtwork(
        uint256 tokenId,
        string memory title,
        string memory artist,
        uint256 price,
        uint256 royaltyPercentage,
        string memory tokenURI
    ) external {
        require(royaltyPercentage <= MAX_ROYALTY, "Royalty too high");

        _mint(msg.sender, tokenId);
```



```

    _setTokenURI(tokenId, tokenURI);

    artworks[tokenId] = Artwork({
        title: title,
        artist: artist,
        price: price,
        forSale: true,
        royaltyPercentage: royaltyPercentage,
        creator: msg.sender
    });

    artistArtworks[msg.sender].push(tokenId);
    totalArtworks++;

    emit ArtworkCreated(tokenId, title, artist, price);
}

// 购买艺术品
function buyArtwork(uint256 tokenId) external payable nonReentrant {
    Artwork storage artwork = artworks[tokenId];
    require(artwork.forSale, "Artwork not for sale");
    require(msg.value >= artwork.price, "Insufficient payment");

    address seller = ownerOf(tokenId);
    address artist = artwork.creator;

    // 计算版税
    uint256 royaltyAmount = (msg.value * artwork.royaltyPercentage) / 10000;
    uint256 sellerAmount = msg.value - royaltyAmount;

    // 转移所有权
    _transfer(seller, msg.sender, tokenId);

    // 支付版税
    if (royaltyAmount > 0) {
        artistRoyalties[artist] += royaltyAmount;
        payable(artist).transfer(royaltyAmount);
        emit RoyaltyPaid(tokenId, artist, royaltyAmount);
    }

    // 支付给卖家
    payable(seller).transfer(sellerAmount);

    // 更新状态
    artwork.forSale = false;

    emit ArtworkSold(tokenId, msg.sender, msg.value);
}

// 设置价格
function setPrice(uint256 tokenId, uint256 newPrice) external {
    require(ownerOf(tokenId) == msg.sender, "Not the owner");
}

```

```

        artworks[tokenId].price = newPrice;
    }

    // 上架/下架
    function toggleForSale(uint256 tokenId) external {
        require(ownerOf(tokenId) == msg.sender, "Not the owner");
        artworks[tokenId].forSale = !artworks[tokenId].forSale;
    }

    // 提取版税
    function withdrawRoyalties() external {
        uint256 amount = artistRoyalties[msg.sender];
        require(amount > 0, "No royalties to withdraw");

        artistRoyalties[msg.sender] = 0;
        payable(msg.sender).transfer(amount);
    }

    // 获取艺术家的作品列表
    function getArtistArtworks(address artist) external view returns (uint256[] memory) {
        return artistArtworks[artist];
    }
}

```

2. 游戏道具NFT

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "@openzeppelin/contracts/token/ERC721/extensions/ERC721URIStorage.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

contract GameItemNFT is ERC721URIStorage, Ownable {
    // 道具信息
    struct GameItem {
        string name;
        string itemType;
        uint256 level;
        uint256 attack;
        uint256 defense;
        uint256 durability;
        uint256 maxDurability;
        bool isEquipped;
    }

    // 状态变量
    mapping(uint256 => GameItem) public gameItems;
    mapping(address => uint256[]) public playerItems;
    mapping(string => uint256) public itemTypeCount;
}

```

```

uint256 public totalItems;

// 事件
event ItemCreated(uint256 indexed tokenId, string name, string itemType, uint256
level);
event ItemEquipped(uint256 indexed tokenId, address indexed player);
event ItemUnequipped(uint256 indexed tokenId, address indexed player);
event ItemRepaired(uint256 indexed tokenId, uint256 newDurability);

constructor() ERC721("Game Items", "GAME") {}

// 创建道具
function createItem(
    uint256 tokenId,
    string memory name,
    string memory itemType,
    uint256 level,
    uint256 attack,
    uint256 defense,
    uint256 durability,
    string memory tokenURI
) external onlyOwner {
    _mint(msg.sender, tokenId);
    _setTokenURI(tokenId, tokenURI);

    gameItems[tokenId] = GameItem({
        name: name,
        itemType: itemType,
        level: level,
        attack: attack,
        defense: defense,
        durability: durability,
        maxDurability: durability,
        isEquipped: false
    });

    playerItems[msg.sender].push(tokenId);
    itemTypeCount[itemType]++;
    totalItems++;

    emit ItemCreated(tokenId, name, itemType, level);
}

// 装备道具
function equipItem(uint256 tokenId) external {
    require(ownerOf(tokenId) == msg.sender, "Not the owner");
    require(!gameItems[tokenId].isEquipped, "Already equipped");
    require(gameItems[tokenId].durability > 0, "Item broken");

    gameItems[tokenId].isEquipped = true;
    emit ItemEquipped(tokenId, msg.sender);
}

```

```

// 卸下道具
function unequipItem(uint256 tokenId) external {
    require(ownerOf(tokenId) == msg.sender, "Not the owner");
    require(gameItems[tokenId].isEquipped, "Not equipped");

    gameItems[tokenId].isEquipped = false;
    emit ItemUnequipped(tokenId, msg.sender);
}

// 使用道具 (减少耐久度)
function useItem(uint256 tokenId) external {
    require(ownerOf(tokenId) == msg.sender, "Not the owner");
    require(gameItems[tokenId].durability > 0, "Item broken");

    gameItems[tokenId].durability--;

    // 如果耐久度为0, 自动卸下
    if (gameItems[tokenId].durability == 0) {
        gameItems[tokenId].isEquipped = false;
    }
}

// 修理道具
function repairItem(uint256 tokenId) external {
    require(ownerOf(tokenId) == msg.sender, "Not the owner");
    require(gameItems[tokenId].durability < gameItems[tokenId].maxDurability, "Already
at max durability");

    gameItems[tokenId].durability = gameItems[tokenId].maxDurability;
    emit ItemRepaired(tokenId, gameItems[tokenId].maxDurability);
}

// 获取玩家的道具列表
function getPlayerItems(address player) external view returns (uint256[] memory) {
    return playerItems[player];
}

// 获取道具属性
function getItemStats(uint256 tokenId) external view returns (
    string memory name,
    string memory itemType,
    uint256 level,
    uint256 attack,
    uint256 defense,
    uint256 durability,
    uint256 maxDurability
) {
    GameItem memory item = gameItems[tokenId];
    return (
        item.name,
        item.itemType,

```

```

        item.level,
        item.attack,
        item.defense,
        item.durability,
        item.maxDurability
    );
}
}

```

ERC1155 应用场景

1. 游戏生态系统

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "@openzeppelin/contracts/token/ERC1155/ERC1155.sol";
import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/security/ReentrancyGuard.sol";

contract GameEconomy is ERC1155, Ownable, ReentrancyGuard {
    // 代币ID定义
    uint256 public constant GOLD_COIN = 1;           // 可替代 - 游戏货币
    uint256 public constant SILVER_COIN = 2;         // 可替代 - 游戏货币
    uint256 public constant COPPER_COIN = 3;         // 可替代 - 游戏货币
    uint256 public constant SWORD = 1001;           // 不可替代 - 武器
    uint256 public constant SHIELD = 1002;           // 不可替代 - 防具
    uint256 public constant POTION = 2001;           // 可替代 - 消耗品
    uint256 public constant SCROLL = 2002;           // 可替代 - 消耗品

    // 道具信息
    struct ItemInfo {
        string name;
        string itemType;
        uint256 level;
        uint256 attack;
        uint256 defense;
        bool isFungible;
        uint256 maxSupply;
    }

    // 状态变量
    mapping(uint256 => ItemInfo) public itemInfo;
    mapping(address => mapping(uint256 => uint256)) public playerStats;
    mapping(address => uint256[]) public playerInventory;

    // 事件
    event ItemCreated(uint256 indexed id, string name, string itemType, bool isFungible);
    event ItemTraded(address indexed from, address indexed to, uint256[] ids, uint256[]
amounts);
    event StatsUpdated(address indexed player, uint256 attack, uint256 defense);
}

```

```

constructor() ERC1155("") {}

// 创建道具
function createItem(
    uint256 id,
    string memory name,
    string memory itemType,
    uint256 level,
    uint256 attack,
    uint256 defense,
    bool isFungible,
    uint256 maxSupply,
    string memory tokenURI
) external onlyOwner {
    require(itemInfo[id].maxSupply == 0, "Item already exists");

    itemInfo[id] = ItemInfo({
        name: name,
        itemType: itemType,
        level: level,
        attack: attack,
        defense: defense,
        isFungible: isFungible,
        maxSupply: maxSupply
    });

    _setURI(id, tokenURI);
    emit ItemCreated(id, name, itemType, isFungible);
}

// 铸造道具
function mintItem(
    address to,
    uint256 id,
    uint256 amount
) external onlyOwner {
    require(itemInfo[id].maxSupply > 0, "Item does not exist");
    require(
        totalSupply(id) + amount <= itemInfo[id].maxSupply,
        "Exceeds max supply"
    );

    _mint(to, id, amount, "");

    // 更新玩家库存
    if (balanceOf(to, id) == amount) {
        playerInventory[to].push(id);
    }
}

// 道具交易

```

```

function tradeItems(
    address from,
    address to,
    uint256[] memory giveIds,
    uint256[] memory giveAmounts,
    uint256[] memory takeIds,
    uint256[] memory takeAmounts
) external nonReentrant {
    require(
        from == msg.sender || isApprovedForAll(from, msg.sender),
        "Not authorized"
    );

    // 先转移要给出的物品
    _safeBatchTransferFrom(from, to, giveIds, giveAmounts, "");

    // 再转移要获得的物品
    _safeBatchTransferFrom(to, from, takeIds, takeAmounts, "");

    // 更新玩家统计
    _updatePlayerStats(from);
    _updatePlayerStats(to);

    emit ItemTraded(from, to, giveIds, giveAmounts);
}

// 使用消耗品
function useConsumable(uint256 id, uint256 amount) external {
    require(balanceOf(msg.sender, id) >= amount, "Insufficient balance");
    require(itemInfo[id].isFungible, "Not a consumable");

    // 销毁消耗品
    _burn(msg.sender, id, amount);

    // 应用效果（这里简化处理）
    if (id == POTION) {
        playerStats[msg.sender][id] += amount * 10; // 恢复生命值
    } else if (id == SCROLL) {
        playerStats[msg.sender][id] += amount * 5; // 增加魔法值
    }

    _updatePlayerStats(msg.sender);
}

// 更新玩家统计
function _updatePlayerStats(address player) internal {
    uint256 totalAttack = 0;
    uint256 totalDefense = 0;

    for (uint256 i = 0; i < playerInventory[player].length; i++) {
        uint256 id = playerInventory[player][i];
        uint256 balance = balanceOf(player, id);
    }
}

```

```

        if (balance > 0 && !itemInfo[id].isFungible) {
            totalAttack += itemInfo[id].attack;
            totalDefense += itemInfo[id].defense;
        }
    }

    playerStats[player][id] = totalAttack;
    playerStats[player][id + 1] = totalDefense;

    emit StatsUpdated(player, totalAttack, totalDefense);
}

// 获取玩家统计
function getPlayerStats(address player) external view returns (uint256 attack, uint256
defense) {
    return (playerStats[player][id], playerStats[player][id + 1]);
}

// 获取玩家库存
function getPlayerInventory(address player) external view returns (uint256[] memory) {
    return playerInventory[player];
}

// 设置URI
function _setURI(uint256 tokenId, string memory tokenURI) internal {
    // 实现URI设置逻辑
}
}

```

2. DeFi协议集成

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "@openzeppelin/contracts/token/ERC1155/ERC1155.sol";
import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/security/ReentrancyGuard.sol";

contract DeFiVault is ERC1155, Ownable, ReentrancyGuard {
    // 代币类型
    uint256 public constant LP_TOKEN = 1;           // 流动性代币
    uint256 public constant REWARD_TOKEN = 2;       // 奖励代币
    uint256 public constant GOVERNANCE_TOKEN = 3;    // 治理代币
    uint256 public constant STAKING_TOKEN = 4;       // 质押代币

    // 质押信息
    struct StakingInfo {
        uint256 amount;
        uint256 startTime;
        uint256 duration;
    }
}

```



```

    uint256 rewardRate;
    bool active;
}

// 状态变量
mapping(address => StakingInfo) public stakingInfo;
mapping(address => uint256) public totalRewards;
mapping(address => uint256) public lastClaimTime;

uint256 public totalStaked;
uint256 public totalRewardsDistributed;

// 事件
event Staked(address indexed user, uint256 amount, uint256 duration);
event Unstaked(address indexed user, uint256 amount, uint256 rewards);
event RewardsClaimed(address indexed user, uint256 amount);

constructor() ERC1155("") {}

// 质押代币
function stake(
    uint256[] memory tokenIds,
    uint256[] memory amounts,
    uint256 duration
) external nonReentrant {
    require(tokenIds.length == amounts.length, "Arrays length mismatch");
    require(duration >= 30 days, "Minimum staking period is 30 days");

    // 转移代币到合约
    _safeBatchTransferFrom(msg.sender, address(this), tokenIds, amounts, "");

    // 计算总质押金额
    uint256 totalAmount = 0;
    for (uint256 i = 0; i < amounts.length; i++) {
        totalAmount += amounts[i];
    }

    // 更新质押信息
    stakingInfo[msg.sender] = StakingInfo({
        amount: totalAmount,
        startTime: block.timestamp,
        duration: duration,
        rewardRate: _calculateRewardRate(duration),
        active: true
    });

    totalStaked += totalAmount;

    // 铸造质押代币
    _mint(msg.sender, STAKING_TOKEN, totalAmount, "");

    emit Staked(msg.sender, totalAmount, duration);
}

```

```

}

// 取消质押
function unstake() external nonReentrant {
    StakingInfo storage info = stakingInfo[msg.sender];
    require(info.active, "No active staking");
    require(
        block.timestamp >= info.startTime + info.duration,
        "Staking period not ended"
    );

    uint256 rewards = _calculateRewards(msg.sender);

    // 销毁质押代币
    _burn(msg.sender, STAKING_TOKEN, info.amount);

    // 转移原始代币
    // 这里需要根据具体的代币类型进行转移

    // 铸造奖励代币
    if (rewards > 0) {
        _mint(msg.sender, REWARD_TOKEN, rewards, "");
        totalRewardsDistributed += rewards;
    }

    // 更新状态
    info.active = false;
    totalStaked -= info.amount;

    emit Unstaked(msg.sender, info.amount, rewards);
}

// 计算奖励
function _calculateRewards(address user) internal view returns (uint256) {
    StakingInfo memory info = stakingInfo[user];
    if (!info.active) return 0;

    uint256 stakingTime = block.timestamp - info.startTime;
    uint256 rewards = (info.amount * info.rewardRate * stakingTime) / (365 days *
100);

    return rewards;
}

// 计算奖励率
function _calculateRewardRate(uint256 duration) internal pure returns (uint256) {
    if (duration >= 365 days) return 15; // 15% APY
    if (duration >= 180 days) return 12; // 12% APY
    if (duration >= 90 days) return 10; // 10% APY
    return 8; // 8% APY
}

```

```

// 领取奖励
function claimRewards() external nonReentrant {
    uint256 rewards = _calculateRewards(msg.sender);
    require(rewards > 0, "No rewards to claim");

    lastClaimTime[msg.sender] = block.timestamp;
    totalRewards[msg.sender] += rewards;

    // 铸造奖励代币
    _mint(msg.sender, REWARD_TOKEN, rewards, "");
    totalRewardsDistributed += rewards;

    emit RewardsClaimed(msg.sender, rewards);
}

// 获取用户奖励
function getPendingRewards(address user) external view returns (uint256) {
    return _calculateRewards(user);
}
}

```